Python Programming & AI Learning Guide

A Comprehensive Personal Study Assistant for B.Tech Computer Science

Project Title:	**AI Personal Study Assistant for Python**
Student Name:	**Rehan M Rafeek**
Department:	**CS AI B**
Target Degree:	**B.Tech Artificial Intelligence & Data Science**
Document Focus:	**Core Fundamentals, Exercises, AI Prompts & Mini Projects**

Academic Year 2026–2027

1. Welcome & How To Use This Guide

Python is a high-level programming language known for readable syntax. It is widely used in software development, automation, data analysis, artificial intelligence, machine learning, web development, and scripting.

HOW TO USE THIS GUIDE

1. Study one topic at a time without rushing.
2. Type every code example yourself and run it in your environment.
3. Try the practice questions before checking the answer key.
4. Utilize the AI Study Assistant prompts provided in Section 13 when stuck.

2. 8-Week Learning Roadmap

Schedule	Topics & Core Concepts covered
Week 1	Setup, print(), variables, data types, input, arithmetic operators
Week 2	if/elif/else decisions, comparison and logical operators
Week 3	for loops, while loops, range(), break and continue statements
Week 4	Data structures: Strings, lists, tuples, sets, dictionaries
Week 5	Functions, parameters, return values, variable scope
Week 6	File handling, exception handling, modules, debugging
Week 7	Object-Oriented Programming (OOP) fundamentals
Week 8	Revision, problem-solving, and mini project building

3. Python Basics

3.1 Your First Program & Variables

A variable is a named container that holds a value. Python automatically detects data types.

```
# First Hello World Program print("Hello, world!") # Declaring Variables name = "Asha" age = 18 course = "Computer Science" print(name, age, course)
```

3.3 Common Data Types

Type	Purpose	Example
int	Whole numbers	`age = 18`
float	Numbers with decimal points	`height = 1.73`
str	Textual character sequences	`name = "Asha"`
bool	Boolean values (True/False)	`is_student = True`
list	Ordered, changeable collection	`marks = [80, 92, 88]`
tuple	Ordered, unchangeable collection	`point = (3, 5)`
dict	Key-value pair collection	`student = {"name": "Asha"}`
set	Collection of unique elements	`numbers = {1, 2, 3}`

3.4 Getting Input & Operators

```
# Reading user input (input always returns a string) age = int(input("Enter your age: ")) print("Next year you will be:", age + 1) # Arithmetic operators: +, -, *, /, // (floor div), % (remainder), ** (power) a, b = 10, 3 print(a // b) # Output: 3 print(a % b) # Output: 1
```

4. Decisions and Loops

4.1 Conditional Statements & Loops

```
# Decisions using if-elif-else mark = 76 if mark >= 90: print("A grade") elif mark >= 75: print("B grade") else: print("Keep practising") # For loop with range(start, end) for number in range(1, 6): print(number) # Prints 1 to 5 # While loop count = 1 while count <= 3: print(count) count += 1
```

5. Collections: Store Multiple Values

5.1 Lists and Dictionaries

Lists store ordered elements accessible by index (starting at 0). Dictionaries store key-value pairs.

```
# List operations subjects = ["Python", "Maths", "Physics"] subjects.append("AI") print(subjects[0]) # Outputs: Python print(len(subjects)) # Outputs: 4 # Dictionary operations student = {"name": "Asha", "mark": 91} print(student["name"]) # Outputs: Asha student["mark"] = 95 # Updates value
```

6. Functions & Scope

Functions are reusable blocks of code that take parameters and return computed outputs.

```
def add_numbers(a, b): return a + b answer = add_numbers(4, 6) print(answer) # Outputs: 10
```

7. Errors & Exception Handling

Common Python errors include `SyntaxError`, `NameError`, `TypeError`, and `IndentationError`.

```
# Exception Handling with try-except try: age = int(input("Enter your age: ")) print("Next year:", age + 1) except ValueError: print("Please enter a valid whole number.")
```

8. Files and Modules

```
# Writing and Reading Files with open("notes.txt", "w", encoding="utf-8") as file: file.write("Practise Python every day.") with open("notes.txt", "r", encoding="utf-8") as file: print(file.read())
```

9. Object-Oriented Programming (OOP)

Classes define blueprints for objects. `__init__` initializes attributes.

```
class Student: def __init__(self, name, mark): self.name = name self.mark = mark def introduce(self): print(f"I am {self.name}; my mark is {self.mark}.") student1 = Student("Asha", 91) student1.introduce()
```

10. Reusable AI Assistant Prompts

- **Learn a Concept:** "Teach me [topic] as a patient Python professor for beginners. Give a simple explanation and code example."
- **Debug Code:** "Help me debug this Python code. Explain the error cause first, then give a hint before showing the solution: [code]"
- **Exam Revision:** "Make concise exam notes on [topic] with definitions, key points, and 5 likely practice questions."

11. Practice Questions

1. What does the `print()` function do?
2. What is the difference between = and ==?
3. What is the output of `print(10 // 3)` and `print(10 % 3)`?
4. Why does `input()` often need `int()` when reading numbers?
5. What values are printed by `range(2, 5)`?
6. Write a program that checks whether a number is positive, negative, or zero.
7. Write a function `square(n)` that returns n squared.

12. Answer Key & Study Checklist

> **Answer Key Highlights**
>
> **1.** Displays output to terminal. | **2.** = assigns, == checks equality. | **3.** Floor division: 3, Modulo: 1. | **4.** `input()` reads text string. | **5.** 2, 3, 4.

13. Final Study Checklist

- [ ] I can explain variables and primary data types.
- [ ] I can use `input()`, `print()`, and type casting.
- [ ] I can write `if/elif/else` conditions and loops.
- [ ] I can create and manipulate lists, dictionaries, and functions.
- [ ] I can handle file I/O and basic exception handling.

Study Principle: Understand it → Type it → Run it → Change it → Explain it.